

1. How work is planned and tracked

- Backlog of record. A single backlog file is the source of truth. Each backlog item references a stable requirement ID introduced in the initial requirements.
- Cadence. One-week iterations aligned with the lab schedule. Each iteration starts with a 10-minute personal planning note and ends with a 10-minute retro note.
- Board. A simple 4-column kanban (Backlog → In Progress → Review → Done). At most two items in In Progress at once.

2. How AI is used inside the workflow

- AI is a partner, not the author. I never accept AI output as final. Every AI artifact must pass the Definition of Done before it moves to Done.
- Allowed roles. Drafting, brainstorming, restating, generating alternatives, finding edge cases, writing tests against my interface, explaining unfamiliar APIs.
- Forbidden uses. Producing code that touches encryption, audit logging, or persistence boundaries without me reviewing line-by-line; introducing new third-party dependencies without satisfying SEC-3.
- Tools. GitHub Copilot Chat in VS Code is the default. ChatGPT and Claude are allowed for longer-form architectural reasoning. Lucid AI is allowed for diagram drafts.

3. How prompts, decisions, and revisions are documented

- Prompt log. Significant prompts are captured in the relevant submission file under a "How AI helped" section, with a note on what was kept, refined, or rejected.
- Architecture Decision Log. Architectural choices and their alternatives; subsequent reversals are appended as new entries rather than overwriting.
- Revision style. I prefer a new commit / new entry over an in-place rewrite, so the history of why a decision changed stays readable.

4. How I decide whether AI output is acceptable

The AI's output is acceptable only if all of the following are true:

1. It traces to at least one stable requirement ID, or it is explicit refactoring work for an item that does.
2. I can defend the design choice in my own words without saying "the AI told me to."
3. It does not introduce an unverified dependency.
4. It does not violate the MVC boundary: the View must not import persistence or encryption modules directly.
5. For security-sensitive code it has been reviewed line-by-line, not just skimmed.
6. It would pass the Definition of Done if a peer reviewed it.

If any check fails, I either re-prompt with stricter constraints or rewrite the section by hand.

5. How I prevent drift, duplication, and low-quality output

- One ID system. All requirements use the FR-* / NFR-* / SEC-* IDs. Renaming requires updating every downstream artifact in the same commit.
- One vocabulary. Class and component names (Note, SecureNote, NoteRepository, NoteService, EncryptionService, PluginManager, SyncService) are fixed in the Architecture Decision Log and reused across all UML, code, and tests.
- No silent regeneration. I do not let the AI re-generate a file from scratch when a targeted edit would do.
- Continuity check before submission. Before each lab is submitted, I scan the prior artifacts for ID and name consistency.

6. Decision-making rules

- Reversible decisions decided in the moment, recorded if non-obvious.
- Hard-to-reverse decisions decided in architecture decisions with a written rationale and a short defense of the rejected alternative.
- Disagreements with the AI are resolved in my favor by default; the AI must convince me, not the other way around.

Reflection

Writing this agreement before producing more deliverables is what will keep AstraNotes from becoming a pile of mutually-inconsistent AI outputs. The two rules I expect to feel the most over the quarter are the stable ID system and the one-vocabulary rule. These two rules cost almost nothing to follow now and would be expensive to retrofit later.